Music Playlist Manager (C++)

Doubly linked list playlist: source code.

```
#include <iostream>
#include <string>
#include <limits>
using namespace std;

// Node representing a song
class Node {
public:
    string title;
    string artist;
    Node* prev;
    Node* next;

    Node(string t, string a) {
        title = t;
        artist = a;
        prev = nullptr;
        next = nullptr;
    }
};

class MusicPlaylistManager {
private:
    Node* head;
    Node* current;
    bool isPlaying;

public:
    MusicPlaylistManager() {
        head = nullptr;
        current = nullptr;
        isPlaying = false;
    }

    // Add a song at the end
void addSong(string title, string artist) {
    Node* p = new Node(title, artist);

    if (head == nullptr) {
        head = p;
        current = p;
    } else {
        Node* q = head;
        while (q->next != nullptr)
            q = q->next;

        q->next = p;
        p->prev = q;
    }

    cout << "Song added successfully.\n";
}

// Delete a song by title
void deleteSong(string title) {
    Node* q = head;

while (q != nullptr && q->title != title)
    q = q->next;

if (q == nullptr) {
    cout << "Song not found.\n";
    return;
}

Node* p = q;

if (p->prev != nullptr)
    p->prev->next = p->next;
else
    head = p->next;
if (p->next != nullptr)
```


Scanned with OKEN Scanner

```
        p->next->prev = p->prev;

    if (current == p)
        current = (p->next != nullptr)
                  ? p->next : p->prev;

    delete p;
    cout << "Song deleted successfully.\n";
}

// Display playlist from first to last
void displayPlaylist() {
    Node* q = head;

    if (q == nullptr) {
        cout << "Playlist is empty.\n";
        return;
    }

    cout << "\nForward Playlist:\n";
    while (q != nullptr) {
        cout << q->title << " - "
             << q->artist << endl;
        q = q->next;
    }
}

// Display playlist from last to first
void displayReverse() {
    Node* q = head;

    if (q == nullptr) {
        cout << "Playlist is empty.\n";
        return;
    }

    while (q->next != nullptr)
        q = q->next;

    cout << "\nReverse Playlist:\n";
    while (q != nullptr) {
        cout << q->title << " - "
             << q->artist << endl;
        q = q->prev;
    }
}

// Play the selected song
void playCurrent() {
    if (current == nullptr) {
        cout << "Playlist is empty.\n";
        return;
    }

    cout << "Now Playing: " << current->title
         << " by " << current->artist << endl;
}

// Move to the next song
void playNext() {
    if (current == nullptr) {
        cout << "Playlist is empty.\n";
    } else if (current->next == nullptr) {
        cout << "Already at the last song.\n";
    } else {
        current = current->next;
        playCurrent();
    }
}

// Move to the previous song
void playPrevious() {
    if (current == nullptr) {
        cout << "Playlist is empty.\n";
    } else if (current->prev == nullptr) {
        cout << "Already at the first song.\n";
```


Scanned with OKEN Scanner

```
    } else {
        current = current->prev;
        playCurrent();
    }
}

// Restart music from the first song
void restartMusic()
{
    if (head == NULL)
    {
        cout << "No songs available!";
        return;
    }

    current = head;
    cout << "Music restarted from the first song.";
}

// Stop the music
void stopMusic()
{
    isPlaying = false;
    cout << "Music stopped.";
}

// Play the complete playlist
void playCompletePlaylist() {
    Node* q = head;

    if (q == nullptr) {
        cout << "Playlist is empty.\n";
        return;
    }

    while (q != nullptr) {
        cout << "Playing: " << q->title
             << " - " << q->artist << endl;
        q = q->next;
    }

    current = head;
}

// Repeat playlist a specified number of times
void repeatAll(int repetitions) {
    if (head == nullptr) {
        cout << "Playlist is empty.\n";
        return;
    }

    if (repetitions <= 0) {
        cout << "Enter a positive number.\n";
        return;
    }

    for (int i = 1; i <= repetitions; i++) {
        cout << "\nRepetition " << i << ":\n";
        Node* q = head;

        while (q != nullptr) {
            cout << "Playing: " << q->title
                 << " - " << q->artist << endl;
            q = q->next;
        }
    }

    current = head;
}

// Show currently selected song
void showCurrent() {
    playCurrent();
}

// Free allocated memory
```


Scanned with OKEN Scanner

```
    ~MusicPlaylistManager() {
        Node* q = head;

        while (q != nullptr) {
            Node* p = q;
            q = q->next;
            delete p;
        }
    }
};

int main() {
    MusicPlaylistManager playlist;
    int choice, repetitions;
    string title, artist;

    do {
        cout << "\n===== MUSIC PLAYLIST MANAGER =====\n";
cout << "1. Add Song\n";
cout << "2. Delete Song\n";
cout << "3. Display Playlist\n";
cout << "4. Display Playlist in Reverse\n";
cout << "5. Play Current Song\n";
cout << "6. Play Next Song\n";
cout << "7. Play Previous Song\n";
cout << "8. Play Complete Playlist\n";
cout << "9. Repeat All\n";
cout << "10. Show Current Song\n";
cout << "11. Exit\n";
cout << "Enter your choice: ";

if (!(cin >> choice)) {
    cin.clear();
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    cout << "Invalid input.\n";
    continue;
}

cin.ignore(numeric_limits<streamsize>::max(), '\n');

switch (choice) {
case 1:
    cout << "Enter song title: ";
    getline(cin, title);
    cout << "Enter artist name: ";
    getline(cin, artist);
    playlist.addSong(title, artist);
    break;

case 2:
    cout << "Enter title to delete: ";
    getline(cin, title);
    playlist.deleteSong(title);
    break;

case 3:
    playlist.displayPlaylist();
    break;

case 4:
    playlist.displayReverse();
    break;

case 5:
    playlist.playCurrent();
    break;

case 6:
    playlist.playNext();
    break;

case 7:
    playlist.playPrevious();
    break;

case 8:
```


Scanned with OKEN Scanner

```
            playlist.playCompletePlaylist();
            break;

        case 9:
            cout << "Enter number of repetitions: ";
            if (cin >> repetitions) {
                cin.ignore(
                    numeric_limits<streamsize>::max(), '\n');
                playlist.repeatAll(repetitions);
            } else {
                cin.clear();
                cin.ignore(
                    numeric_limits<streamsize>::max(), '\n');
                cout << "Invalid repetition count.\n";
            }
            break;

        case 10:
                playlist.showCurrent();
                break;

            case 11:
                cout << "Exiting Music Playlist Manager.\n";
                break;

            default:
                cout << "Invalid choice. Try again.\n";
            }

        } while (choice != 11);

        return 0;
    }
```


Scanned with OKEN Scanner

```
==== MUSIC PLAYER ====
1. Add Music
2. Delete Music
3. Display Playlist
4. Play Music
5. Next Song
6. Previous Song
7. Restart Music
8. Stop Music
9. Reverse Playlist
10. Exit
Enter your choice: 1
Enter song name: tu hai mera
Enter singer name: pritam

Music added successfully!
```

Scanned with OKEN Scanner